Kristianstad University
SE-291 88 Kristianstad
+46 44-250 30 00
www.hkr.se

Independent project (degree project), 15 credits, for the degree of
Bachelor of Computer Science
Spring Semester 2026
Faculty of Computer Science

A Comparative Study of Time-Efficiency in Linear Regression and Decision Tree Using Python and C# libraries

Kotayba Sayed

Simon Persson

Authors

Simon Persson

Kotayba Sayed

Title

A Comparative Study of Time-Efficiency in Linear Regression and Decision Tree Using Python and C# libraries

Supervisor

Shahbazi Zeinab

Assessing teacher

Kamilla Klonowska

Examiner

Kamilla Klonowska

Abstract

This thesis compares Python and C# with respect to execution time for a linear regression as well as decision tree machine learning workflow implemented using established libraries in each ecosystem. A controlled, quantitative paired-measurement experiment is conducted where both implementations use the same dataset, feature selection, train–test split strategy, and are executed on identical hardware and operating system settings. Execution time is measured using high-resolution timers and decomposed into preprocessing, training, inference, and total workflow time. To evaluate scalability, the dataset size is varied via systematic subsampling (e.g., 5k–50k records) while preserving feature distributions, and each configuration is repeated to reduce measurement noise. The study reports workflow-level timing results and analyzes which workflow stages contribute most to any observed differences, providing practical guidance on execution time behavior when using standard libraries under realistic conditions.

Keywords

Machine Learning, Libraries, Programming Ecosystem Comparison, Python, C#, Decision Tree, Linear Regression, Time efficiency, Model Complexity.

Content

1. Introduction

1.1 Background

In software development and machine learning, the choice of programming ecosystem can influence how time-efficiently a machine learning workflow runs. Ecosystems in Python and C# offer different strengths depending on factors such as execution speed, available libraries, and ease of use. Understanding how these ecosystems perform under similar conditions becomes especially important when working with larger datasets, where execution time can become a practical limitation.

Programming languages also differ in how they execute code, which may influence computational performance. Python typically relies on an interpreted execution model, offering flexibility but often introducing execution time overhead. C# is compiled into an intermediate language that is optimized before execution, which can result in faster and more predictable performance. At the same time, performance in machine learning workflows is shaped not only by the language itself, but also by the libraries and execution time environments through which models are implemented and executed. These differences make Python and C# suitable subjects for comparison when evaluating execution time under identical conditions[9].

Computer science is broadly defined as the study of computers and computational systems, including their theoretical and algorithmic foundations, the processing of information in digital systems and the design and application of hardware and software[2][3][14]. Meanwhile, software engineering is broadly defined as the branch of computer science that focuses on the design, development and maintenance of software application, while applying engineering principles to create efficient, reliable, and user oriented software solutions[4][5][6]. These definitions provide the disciplinary context for the thesis. The study relates to computer science through its focus on computational systems and performance, and to software engineering through its practical comparison of software based machine learning implementations in Python and C#.

1.2 Research Problem and Motivation

A common assumption is that compiled languages outperform interpreted languages in execution speed [12]. However, the literature does not consistently report reproducible, end-to-end comparisons that evaluate Python and C# as complete machine-learning workflows under matched conditions (dataset size is varied through subsampling while maintaining identical preprocessing and split strategies, equivalent preprocessing decisions, same hardware and OS), and many discussions instead focus on broader language characteristics or high-level ecosystem considerations rather than paired ML workflow timing using standard libraries [9][10][13]. When comparisons are not controlled, it becomes difficult to attribute observed differences to the language ecosystem rather than external factors [10][21].

The motivation for this thesis is therefore to perform a fair, reproducible comparison grounded in measurable execution time [10][21]. Since developers typically use established libraries rather than custom implementations in real projects, the comparison should reflect typical usage and focus on end-to-end workflow time rather than isolated microbenchmarks [10]. To strengthen the practical relevance and generalizability of the study, the comparison includes both linear regression and decision tree regression, allowing performance to be examined across algorithm families with different computational characteristics.

In addition, machine learning is not only “model training.” Real workflows include preprocessing and data handling steps that can dominate execution time depending on framework behavior and hardware utilization. Research that profiles preprocessing workflows shows that these stages can be major contributors to performance and should be examined explicitly rather than treated as a single black box [21]. This motivates looking beyond total execution time and also measuring where time is spent. The study is further motivated by the need to interpret observed differences in greater depth by examining relevant library behaviour, execution time characteristics, and selected implementation-level details in the python and C# ecosystems.

1.3 Aim and Objectives

Aim:

The aim of this thesis is to compare Python and C# with respect to execution time when training and running linear regression as well as decision tree regression using established machine learning libraries under controlled conditions, and to interpret observed differences in relation to execution time and library implementation characteristics in each ecosystem.

Objectives:

To achieve the aim, the thesis will:

1. Implement equivalent linear regression and decision tree workflows in Python and C# using commonly adopted libraries in each ecosystem.
2. Ensure a controlled experimental setup by keeping non-language factors constant (e.g., dataset, feature selection, train–test split, hardware, and OS).
3. Measure execution time in milliseconds for the workflow, with repeated runs to reduce random variation.
4. Break down execution time into workflow parts (e.g., preprocessing, training, inference) to identify where differences arise.
5. Evaluate how performance differences change as dataset size increases (using consistent subsampling steps).
6. Interpret measured differences in relation to execution time behavior (e.g., JIT compilation, interpreter execution) and library implementation characteristics.
7. Examine whether observed performance differences remain constant across two different algorithm families or vary depending on algorithm characteristics.
8. Support the interpretation of results through targeted implementation-level analysis of relevant library behaviour, execution time characteristics, profiling evidence, and selected source-code paths in the python and C# ecosystems.

1.4 Research Question

RQ1 (Main):

How do Python and C# differ in execution time across machine learning workflow stages including preprocessing, training, and inference, using established machine learning libraries under controlled conditions, and how can the observed differences be interpreted in relation to library and execution time implementation characteristics?

RQ2 (Sub):

Which parts of the machine learning workflow contribute most to execution time differences (e.g., preprocessing, training, inference), and what do these phase level differences suggest about library/workflow overhead in each ecosystem?

RQ3 (Sub):

How does the performance difference change as dataset size increases, and what does this indicate about fixed overheads versus scaling related implementation costs in the compared libraries/execution times?

RQ4 (Sub):

Do Python and C# exhibit consistent performance advantages across different algorithm families or do differences depend on algorithm characteristics such as split search complexity, branching behaviour and preprocessing requirements?.

Reasoning for the sub-questions:

RQ2 is included because workflow stages can have different bottlenecks; profiling research shows preprocessing and framework/hardware interaction can significantly affect execution time, meaning total time alone may hide the real cause of differences [21]. RQ3 is included because execution time behaviour can change as the workload grows; evaluation methods and repeated validation can have substantial computational cost, motivating explicit consideration of how performance scales rather than assuming that small-scale results generalize directly to larger datasets [22]. RQ4 is included because the study compares both linear regression and decision tree regression, which represents different algorithm families with different computational characteristics. This makes it relevant to examine whether any observed performance is consistent across models or depends on the structure and complexity of the algorithm being used[26].

1.5 Contributions to Computer Science

This thesis contributes a controlled and reproducible comparison of Python and C# as machine learning ecosystems using linear regression and decision tree regression under matched experimental conditions. By evaluating equivalent workflows implemented with established libraries, the study provides evidence-based executing-time results rather than assumptions about language performance.

The thesis contributes by reporting phase level execution time measurements-including preprocessing, training, inference, and total workflow time rather than only total execution time. This gives a more detailed view of

where time is spent in machine learning workflow and supports software engineering principles of measurement, transparency and reproducibility. The study also contributes by analysing how execution time differences change as dataset size increases, which improves understanding of scalability and practical performance under larger workloads.

In addition, the thesis goes beyond a single-model comparison by including both linear regression and decision tree regression, making it possible to examine whether performance differences remain consistent across different algorithm families or depend on algorithm characteristics. Finally , the thesis provide a deeper explanatory behaviour, library implementation characteristics, profiling evidence and selected library execution paths in the Python and C# ecosystem-level machine learning performance, but also a more detailed interpretation of why performance differences may occur under which conditions.

1.6 Structure of the Thesis

This thesis begins with an introduction that outlines the background, research problem, aim, objectives, and research questions. Chapter 2 presents the literature review and discusses prior work relevant to programming-language performance and machine learning workflow profiling. Chapter 3 describes the methodology, including the experimental design, dataset handling, and timing approach used to ensure a controlled comparison. Chapter 4 describes the implementation of the workflows in Python and C#. Chapter 5 presents the results and analysis. Finally, Chapter 6 discusses the findings, limitations, and concludes the thesis with suggestions for future work.

2. Literature review

2.1 Identifying Relevant Literature

For this thesis, keywords are selected which are Machine Learning, Libraries, Programming Ecosystem Comparison, Python, C#, Decision Tree, Linear Regression, Execution Time, Model Complexity, Compiled vs Interpreted.

These keywords were selected because they represent the programming languages being compared, highlight the focus on time-efficiency (execution time), and reflect the broader topic of measuring performance differences between programming environments in machine learning workflows. Since this thesis compares a compiled and an interpreted language, C# and Python were chosen to represent these categories and define the range of the research.

To find suitable literature and articles, the search keywords were used in different combinations such as “Python vs C# performance”, “Compiled vs interpreted languages” , “machine Learning execution time” , “linear regression implementation”, “decision tree”, “languages performance standard” and related terms. Searches were carried out using academic databases such as IEEE Xplore, Google Scholar and Kristianstad University Library using Super search. The articles and literature are stored in Zotero.

2.1.1 Inclusion and Exclusion Criteria

After collecting candidate sources, inclusion and exclusion criteria were applied to ensure that the selected literature was relevant to the scope of the thesis and suitable for supporting a fair interpretation of results. The criteria were designed to prioritize research that provides measurable performance evidence, relates to machine learning workflows, and includes sufficient methodological detail to allow evaluation of validity and comparability.

Studies were included if they discussed or measured execution time or runtime performance, focused on Python, C#, .NET, or programming-language execution models relevant to cross-language comparisons, or examined machine learning workflows such as preprocessing, workflow profiling, training, inference, or end-to-end execution time measurement. Preference was given to studies that used established libraries or frameworks, reflecting typical real-world machine learning development rather than purely custom implementations. Additionally, studies were required to report sufficient methodological details, such as hardware and software context, dataset characteristics, and measurement approaches, to allow interpretation and comparison of results.

Sources were excluded if they primarily discussed language popularity, developer opinions, or conceptual comparisons without measurable runtime or execution-time evidence. Studies unrelated to machine learning workflows or that did not provide insights into execution-time behaviour in machine learning contexts were also excluded. Research using non-comparable tasks, lacking experimental control, or providing insufficient methodological detail was not considered suitable. In addition, studies focusing mainly on GPU-based deep learning training were excluded when their results could not reasonably transfer to the CPU-based linear regression and decision tree workflows used in this thesis, except when they were referenced for general benchmarking methodology or contextual discussion.

2.2 Relevant Theories and Models

The thesis is based on two major models: the code execution model of programming languages and the machine learning workflow model, represented here as a linear regression and decision tree regression workflow. These two perspectives form the theoretical foundation for understanding how Python and C# can differ in execution time behaviour, and how those differences influence end-to-end workflow time.

2.2.1 Code Execution Model

The code execution model describes how programming languages execute code. C# is typically compiled to intermediate language (IL) and executed under the .NET Common Language Runtime (CLR), where just-in-time (JIT) compilation and runtime optimizations can improve execution performance. Python, in its default Python implementation, compiles source code to bytecode that is executed by an interpreter, while C# is compiled to intermediate language and optimized through JIT compilation. These execution-model differences are often used to explain why compiled or JIT-compiled environments may outperform interpreted execution on compute-heavy tasks. [12]

At the same time, ML performance is not only determined by the language core. In Python, many ML computations are executed inside optimized native libraries and backends, which can reduce the impact of interpreter overhead in practice. This means that performance results often reflect the whole ecosystem (execution time + libraries + underlying native code), rather than only “interpreted vs compiled.” [10]

Evidence from-cross ML experiments supports the view that observed execution time is often an ecosystem effect rather than a simple “interpreted vs compiled” outcome. Salihu and Tafa implement two equivalent deep learning model(CNN and CapsNet) in python and c# and deliberately use the same underlying framework (CNTK) in both languages to improve comparability.
Their results show that computational efficiency can vary by model even under comparable conditions C# is faster for their CNN case(epoch time /images-per-second), while python is faster for their CapsNet case, indicating that performance differences are not fixed by language alone and motivating empirical measurement rather than assumptions[23]. For this thesis, the study provides a methodological motivation to treat execution time as an ecosystem-level outcome and to measure it directly rather than relying on general expectations about compiled versus interpreted execution.

2.2.2 Machine Learning Workflow Model (Linear Regression and Decision tree as a Baseline)

The machine learning models used are linear regression and decision tree. Linear regression is chosen because it is a well-established method and still involves meaningful numerical computation. Decision trees introduce branching logic, split search procedure and potentially different memory and cache behaviours. Including both a linear and a tree based model enables evaluation of whether observed ecosystem level performance differences persist across fundamentally different training dynamics and computational patterns. Decision trees are commonly used in time and throughput constrained settings where the cost of applying the model matters in practice. Buschjäger and Morik study decision tree and random forest implementations for fast filtering and emphasize that in such scenarios, model application (inference) and implementation strategy strongly influence performance and resource use.[24] In this thesis, the focus is not on predictive accuracy, but on execution time in a realistic workflow (preprocessing, training, inference, and total workflow time).

A key reason to treat ML as a workflow rather than only “training” is that real workflows include data loading and preprocessing steps that can be major execution time contributors depending on framework behavior and hardware utilization. Workflow profiling research shows that preprocessing can be a performance bottleneck and that bottlenecks may differ across workflow stages and configurations, which supports measuring workflow stages separately rather than timing only a single block. [21]

Finally, execution time evaluation itself can become expensive as workloads grow, especially when evaluation procedures are repeated (e.g., cross-validation). Work on approximating cross-validation in kernel-based models illustrates how repeated evaluation can scale poorly with dataset size and motivates considering scalability explicitly when designing and interpreting execution time experiments. [22]

2.3 Critical Review of Previous Works

Previous research provides useful insights into programming language performance and ML execution, even though many studies approach the topic differently than this thesis.

Choudhary et al. conduct a comparative experimental study analyzing the behaviour of several machine learning algorithms, including linear regression and

decision trees [25]. Their work evaluates algorithm performance through experimental comparison and demonstrates that different algorithms can produce different results depending on implementation and dataset characteristics. However, the study primarily focuses on predictive accuracy rather than execution time, and therefore does not analyze how programming language implementations affect execution time performance.

Zhu also compares the performance of linear regression and decision tree algorithms through an experimental study. The results show that decision trees can provide more accurate predictions in certain datasets. While the study highlights performance differences between algorithms, it mainly evaluates prediction accuracy rather than execution time or programming language implementation differences [26].

Alomari et al. present a broad comparative study of six programming languages and include both qualitative criteria and limited execution time-oriented experiments (e.g., database read/write operations). Their results indicate that execution-time behavior can differ across languages, supporting the general idea that language choice can matter for execution time. However, the study is not centered on controlled end-to-end machine learning workflows using standard ML libraries under matched experimental conditions, so its findings mainly serve as contextual motivation rather than direct evidence for ML workflow performance. [9]

Castro et al. focus on high-performance Python in data science and ML, showing that Python performance often depends on optimized libraries and compiled extensions. This is relevant because it supports the idea that Python execution time results should be interpreted as “ecosystem performance” rather than only interpreter limitations. [10] Youssef similarly discusses Python’s dominance in ML and highlights that performance trade-offs depend on frameworks and alternatives, reinforcing that execution time comparisons must consider library and ecosystem behaviour. [8]

Bahar et al. provide a survey of language features and comparisons involving Python, Java, and C#, which helps frame execution time and ecosystem differences between languages. [11] Swacha and Muszyńska compare Python and C# from a student perspective, which is useful contextually, but it is not designed as a controlled ML workflow benchmark. [13]

Ahmad et al. investigate flaky test detection in software projects using machine learning across multiple programming languages. Their experimental methodology demonstrates that inconsistent results can occur across repeated executions, highlighting the importance of repeated runs and controlled

experiments when evaluating machine learning systems. Although their research focuses on software testing rather than execution time performance, it supports the methodological decision in this thesis to perform repeated measurements to ensure reliable execution-time results [27].

Other studies support the general expectation that compiled languages often perform better in execution time for computational workloads. Zhulkovskyi et al. report computational performance comparisons across modern languages (outside ML), which provides theoretical support for expecting time differences and motivates testing whether similar patterns appear in ML workflows. [12]

Salihu and Tafa provide a direct example of a python vs C# computational comparison in an ML workload. They benchmark CNN and CapsNet on MNIST and report timing focused metrics including time per epoch and images processed per second, alongside loss/accuracy. A methodological strength is that they use CNTK as a shared library in both implementations, improving fairness by reducing “different backend” bias, and they report their hardware context[23]. However, the work focuses on deep learning training on a GPU-enabled setup and does not target classical CPU-based regression models or workflow preprocessing time, so it mainly supports this thesis as (1) evidence that cross language performance differences exist in ML workload and (2) an example of how to design a more comparable benchmark.

Harsh et al. evaluate ML defect prediction models for Python software. This supports the idea that ML workflows are common and library-driven in real development contexts, but it does not provide a controlled cross-language comparison with C# under identical conditions. [7]

A key addition for this full thesis scope is workflow-focused work. Bachkaniwala et al. (Lotus) show that preprocessing workflows can be a major execution time contributor, and they profile workflows across frameworks and hardware. This is directly relevant to this thesis because it supports measuring time in workflow parts (preprocessing, training, inference) rather than only reporting total execution time. [21]

Zhang et al. benchmark multiple deep learning libraries and show that inference performance is highly dependent on the software stack: the best performing library is fragmented across different model-hardware combinations, meaning no single library is consistently fastest. To explain bottlenecks rather than only report timing, they also “dive into the source code” (e.g., for NCNN) and break cold star inferences into workflow steps, finding that memory preparation accounts for most of the cold-start overhead and is often implemented single threaded, limiting the benefit of multi core hardware. although their domain is mobile deep learning,

the study is relevant to this thesis because it demonstrates a concrete, reproducible approach for interpreting execution time differences by combining benchmarking with targeted inspection of implementation paths, supporting the planned library/execution time analysis in the Python and C# workflows.[28]

Buschjäger and Morik study how different implementations of decision trees and random forests affect throughput and resource usage, comparing CPU (von-Neumann) execution with FPGA-based designs for fast sensor-data filtering. They drive theoretical cost models (in clock cycles/resources) for multiple implementation strategies (e.g., naive traversal versus unrolled if-else structure and SIMD-oriented layout) and validate them experimentally, reporting throughput results with repeated trials. Although the work is not a Python vs C# comparison and focuses on embedded classification/filtering rather than regression, it is relevant to representation and execution strategy, supporting the idea that library implementation choices can materially influence measured inference time.[24]

2.4 Gaps in Existing Research

A review of the selected literature shows several clear gaps that motivate this thesis.

Authors/ Year	Study focus	Method/context	Main relevance to this thesis	Gap/ limitation relative to this thesis
V. J. Reddi et al. (2020)	Standardized benchmarking of ML inference systems.	Benchmarking methodology for ML inferences across different hardware, software, framework and deployment scenarios. The study introduces standardized rules, representative workloads and realistic evaluation scenarios to improve reproducibility and	Supports the thesis methodologically by showing that ML performance comparisons require controlled, reproducible benchmarking conditions and comparable evaluation criteria rather than informal speed comparisons. It strengthens the	Focuses on inference benchmarking across ML systems broadly, not on a controlled comparison between Python and C# using classical regression

		compatibility .	argument for keeping dataset, setup and timing procedure consistent.	workflows and phase level workflow timing.
C.Zhang et al. (2021)	Machine learning benchmark suite for CPU-GPU integrated architectures.	Experimental benchmark-suite study examining ML workload behavior on integrated utilization. The paper argues that understanding ML performance on such architectures is still an open problem and proposes a benchmark suite for that purpose.	Shows that current ML performance literature often evaluates performance as a systems and architecture problem, where hardware behaviour and execution environment shape measured results. This supports the thesis approach where execution time must be interpreted to more than the algorithm	The paper is hardware oriented and does not compare Python and C# or standard libraries such as scikit-learn and ML.NET in a matched classical ML workflow.
Y.Cheng et al. (2021)	End to end deep learning workflow acceleration, especially preprocessing and scheduling.	Workflow-level experimental study of deep learning systems that identifies preprocessing as a major bottleneck and investigates how data preprocessing and computational scheduling affect end-to-end performance. The paper explicitly states that prior work often focuses on training/inference while ignoring other workflow stages.	Directly supports the thesis decision to measure preprocessing, training, inference and total workflow separately. It strengthens the argument that total execution time alone may hide where performance differences actually arise	Focuses on deep learning workflow on specialized hardware rather than CPU-based linear regression and decision tree workflows in Python and C#.
M. Paganelli	Performance of end to end	Experimental evaluation of trained	Very relevant because it included	Focuses on in-DBMS

et al. (2023)	ML pipelines executed inside DBMS environments.	ML pipelines containing both featurizers and models, translated into SQL and compared against SKlearn and ML.NET across different tasks, datasets and execution settingML as a full workflow involving data preparation and serving rather than only model itself.	classical ML pipelines, explicitly considers full pipelines with featurizers and models and compares performance against both SKlearn and ML.NET. This supports the thesis framing of ML performance as ecosystem and workflow dependent.	prediction serving and SQL execution context rather than a controlled Python vs C# comparison of preprocessing, training, inference and total execution time under identical hardware and dataset conditions.

(Table 1)

As table 1 shows there is a clear theme in recent IEEE literature is that machine learning performance is increasingly studied as a workflow and systems level issue rather than only as a property of the learning algorithm itself. Recent work emphasizes the importance of standardized benchmarking and reproducible evaluation across different software and hardware settings, showing that fair comparison requires carefully defined workloads, scenarios and measurement rules. Other studies show that performance can be strongly affected by the execution environment, including hardware architecture, memory behaviour, scheduling and deployment setting. In addition, current literature highlights that preprocessing and other non-training stages can be major contributors to the total runtime meaning that end to end workflow is often more informative than measuring training time alone. At the same time, despite this broader system oriented perspective the literature still provides limited evidence from controlled cross language comparisons of classical machine learning workflows using the same dataset, preprocessing decisions, train-test split, hardware and operating system settings. This helps explain the gap addressed in the present thesis, which compares Python and C# under matched conditions and measures preprocessing, training, inference and total workflow time separately.

First, many studies that compare programming-language performance focus on general algorithms or numerical methods rather than controlled machine learning workflows using the same dataset and setup. This makes it uncertain whether

observed language performance patterns transfer to realistic ML workflows. [9][12]

Second, several studies discuss machine learning performance inside Python without comparing Python against a compiled language like C# using identical workflow decisions. This means there is still limited evidence about how the same ML workflow behaves across these two ecosystems when run under matched conditions. [7][10]

Third, studies that compare Python and C# often focus on language characteristics, education, or broad surveys rather than reproducible benchmarking of ML workflow execution time using standard libraries. This limits their usefulness for answering execution-time questions in ML contexts. [11][13]

While there is some IEEE evidence of cross-ecosystem ML benchmarking between Python and C#(e.g., using a shared framework to improve comparability), existing comparisons are often conducted on deep learning image-classification workloads rather than classical regression workflows. This leaves uncertainty about whether similar ecosystem-level time differences appear in CPU-based linear regression and decision tree regression workflows, especially when execution time is broken down in preprocessing, training and inference stages[23].

Fourth, many performance discussions do not clearly break down end-to-end workflow time into workflow phases. Workflow profiling research suggests this matters because preprocessing and data-handling can dominate execution time and hide where differences truly come from if only total time is measured. [21]

Finally, scalability and repetition cost are often not treated as a central part of cross-language ML performance comparisons. Literature on the computational cost of repeated evaluation supports the importance of explicitly considering how execution time behaviour changes as workload size grows. [22]

Taken together, existing research does not provide a controlled and fair execution time comparison of Python and C# for the same machine learning workflow under identical conditions while also reporting workflow-stage timing and dataset scaling. This thesis aims to fill these gaps by conducting a paired, controlled experiment using established libraries, measuring execution time for preprocessing, training, inference, and total workflow execution time, and evaluating how differences change as dataset size increases. [21][22]

2.5 Summary of literature review

3. Research Methodology

3.1 Research Design

This thesis uses a comparative experimental research design with a quantitative approach. The purpose is to measure how Python and C# differ in execution time when running the same machine learning task under controlled conditions.

The study is designed as a controlled benchmarking experiment, with paired measurements, meaning identical machine learning workflows are implemented in both Python and C#, using the same dataset size, same selected variables, same target variable, and the same train–test split 80/20 strategy with random seed 42. The objective is to measure execution time across comparative phases of the workflow while controlling for dataset size, algorithm type and hardware environment. This reduces the effect of external factors and makes it easier to attribute measured differences to the programming language ecosystem rather than changes in the setup. [9][12]

The machine learning process is treated as a multi-stage workflow, and is measured in separate phases: pre-processing time (+split time), training time, inference time, and total workflow time. This is done because machine learning execution time is not only affected by training; profiling research shows that preprocessing and workflow stages can dominate execution time depending on framework behaviour and hardware usage. [21]

To study scalability, the experiment is repeated with multiple dataset sizes created through systematic subsampling of the same dataset (for example 5k, 10k, 25k, and 50k records), while preserving feature distributions. No synthetic data is created. [20]

3.2 Tools and Technologies Used

The study is conducted using the same hardware and operating system for both Python and C# experiments to ensure fairness and reproducibility.

Hardware and Operating System

- CPU: Intel i5-12600KF (10 cores, 16 threads)

- RAM: 32 GB
- Storage: 930 GB SSD
- OS: Windows 11 Pro

Software and Libraries

- Python 3.12 (Visual Studio 2022, scikit-learn, NumPy).
- C# 14 (.NET, Visual Studio 2022, ML.NET).
- All software versions remain constant during testing, and background processes are kept at a minimum to reduce interference with timing results.

Execution time in Python is measured with perf_counter_ns, which is the most precise available, measuring in nanoseconds. In C#, the method used for measuring time is Stopwatch. Results are stored for later analysis.

3.3 Algorithms or Frameworks

The algorithms used are Linear Regression and Decision Tree, using the imported (standard) library implementation in each ecosystem. This choice reflects typical developer usage, where performance is experienced through the full ecosystem (language execution time + library + underlying numerical backend), rather than through a custom hand-coded model. [10]

The workflow is kept equivalent in both languages and models:

1. Load the dataset once, to avoid mixing disk cache effects
2. Preprocess and prepare features and target variables.
3. Split into training and test sets using the same split strategy and seed.
4. Train (fit) the machine learning model.
5. Run inference (predict) on the test set.
6. Record phase times and total workflow time

3.4 Data Variables

The dataset was loaded and cleaned once before running the experiments. This is in order to primarily measure the execution-time of the algorithm, not the file system performance, therefore reducing external system overhead. Loading the dataset once also helps avoid caching issues that may otherwise occur. This strengthens the internal validity.

Dependent Variables (measured in nanoseconds)

- Pre-processing time
- Training time
- Inference time
- Total workflow time

Independent Variables

- Programming language (Python, C#)
- Model implementation (standard default imported library)
- Dataset size

Controlled Variables

- Dataset (same source, same selected variables, same target variable)
- Hardware and OS
- IDEs and software versions
- Train–test split strategy (ratio + seed kept consistent)
- Number of repetitions per configuration

This setup ensures the comparison is fair and that execution time differences are not caused by different data, splits, software versions, or machine conditions. [9][12]

3.5 Data Collection and Processing

Dataset

The dataset used is the LinkedIn Compatibility Dataset: 50K profiles in CSV format, which size is ~1.03 GB. The dataset originates from Kaggle, and contains synthetic compatibility scores between different LinkedIn profiles. The dataset can be managed using regression because the target variable is continuous.

To ensure that the measurements were statistically meaningful, each experiment was repeated 100 times. For each dataset size, both machine learning models were executed repeatedly using the same dataset subset. By repeating the experiment, the impact of operating system scheduling, memory allocation and runtime initiation effects.

The variables used are:

- skill_match_score
- skill_complementarity_score
- network_value_a_to_b
- network_value_b_to_a
- career_alignment_score

- experience_gap
- industry_match
- geographic_score
- seniority_match
- compatibility_score

3.5.1 Pre-processing

Pre-processing is performed using the same rules in both environments to avoid introducing bias. The preprocessing includes steps such as:

- ensuring consistent column types,
- handling missing values using the same approach,
- removing duplicates if they exist,
- extracting the feature columns and the target column, and preparing the data structures required by each framework.

3.5.2 Dataset-size scaling (subsampling)

In order to ensure scalability, the dataset size is varied through deterministic subsampling of the original dataset using fixed random seed, where the sizes are 100, 75, 50, 25 and 10% of the original file size, while preserving feature distributions to keep subsets representative. No synthetic data is created. This allows us to analyze how the execution time scales as the dataset grows.

3.5.3 Timing procedure and repetitions

Execution time is measured using a high-resolution monotonic timer. Each phase of the workflow is measured separately, including the dataset splitting, model training and inference. The time values are then recorded in nanoseconds for high precision and stored for later analysis.

Execution time is measured in phases:

- Pre-processing time: from loading the dataset until it is ready for training
- Training time: the model fitting step
- Inference time: prediction on the test set
- Total workflow time: from dataset loading start to inference completion

Each configuration is executed 100 times and summarized (e.g., average/median) to reduce random variation from OS scheduling, caching, and execution time warm-up effects. Phase-level timing is included because workflow profiling research shows that workflow stages can contribute unevenly to total execution time and may hide bottlenecks if only total time is measured. [21]

3.5.4 Model selection

Linear regression and decision tree were selected as they represent two widely used machine learning approaches, and they both have different computational characteristics. Linear regression primarily uses linear algebra operations, while decision trees use recursive partitioning and branching. This leads to a more diverse and interesting result to analyze

3.6 Method Justification

The methodology choices are made to ensure a fair and reproducible comparison aligned with the research questions. A controlled quantitative experiment is appropriate because the thesis aims to measure execution time differences under identical conditions rather than relying on subjective judgement. [9][12] Using standard imported libraries reflects real-world practice and evaluates performance as it would be experienced by developers in each ecosystem. [10] Measuring workflow phases (preprocessing, training, inference) is justified because ML workflows often spend significant time outside training, and profiling research shows preprocessing can be a dominant factor. [21] Varying dataset size is necessary to examine scalability and understand whether performance differences change as workload increases. [20] This methodology therefore supports a controlled comparison of Python and C# execution time for a linear regression and decision tree workflow, without relying on any accuracy metric such as R^2, and directly targets the thesis research questions.

For each of the experimental configurations, the recorded execution times were aggregated using descriptive statistics including mean, median and standard deviation. These statistics provide a stable estimate of execution time, and also reduce the influence of overhead noise.

3.7 Threats to validity

In order to reduce the threats to validity of this study, this study is ensuring the internal validity by using identical dataset, algorithms and hardware. External validity may have results that differ due to GPU workloads and deep learning frameworks.

4.Experiment Implementation

5. Results and Analysis

6. Discussion

7. Conclusion and Future Work

7.1 Summary of Key Findings

7.2 Contributions of the Study

7.3 Recommendations for Future Research

8. References

1. *Dictionary.com | Meanings & Definitions of English Words*. (2025, November 14). Dictionary.Com. https://www.dictionary.com/
2. Michigan Tech. (2024). *What Is Computer Science?* Michigan Technological University; Michigan Technological University. https://www.mtu.edu/cs/what/
3. *Computer science | Definition, Types, & Facts | Britannica*. (2025, October 17). https://www.britannica.com/science/computer-science
4. *What Is a Software Engineer? | Skills and Career Paths*. (n.d.). Retrieved 14 November 2025, from https://www.computerscience.org/careers/software-engineer/e
5. Imed Bouchrika. (2025). *What Does a Software Engineer Do: Responsibilities, Requirements, and Salary for 2025*. Research.com. https://research.com/advice/what-does-a-software-engineer-do-responsibilities-requirements-and-salary
6. Michigan Tech. (2024). *What is Software Engineering? | Computer Science | Michigan Tech*. Michigan Technological University. https://www.mtu.edu/cs/undergraduate/software/what/
7. Harsh, Santoshi, H., & Singh, P. (2025). Comparative Study of Machine Learning Based Defect Prediction Models for Python Software. *2025 6th International Conference on Inventive Research in Computing Applications (ICIRCA)*, 1867–1872. https://doi.org/10.1109/ICIRCA65293.2025.11089647
8. Youssef, J. (n.d.). *Python's Dominance in Machine Learning: Unraveling its Emergence and Exploring the Trade-offs of Faster Alternatives*.

9. Alomari, Z., Halimi, O. E., Sivaprasad, K., & Pandit, C. (2015). *Comparative Studies of Six Programming Languages* (No. arXiv:1504.00693). arXiv. https://doi.org/10.48550/arXiv.1504.00693
10. Castro, O., Bruneau, P., Sottet, J.-S., & Torregrossa, D. (2023). Landscape of High-Performance Python to Develop Data Science and Machine Learning Applications. ACM Comput. Surv., 56(3), 65:1-65:30. https://doi.org/10.1145/3617588
11. Bahar, A. Y., Shorman, S. M., Khder, M. A., Quadir, A. M., & Almosawi, S. A. (2022). Survey on Features and Comparisons of Programming Languages (PYTHON, JAVA, AND C#). *2022 ASU International Conference in Emerging Technologies for Sustainability and Intelligent Systems (ICETSIS)*, 154–163. https://doi.org/10.1109/ICETSIS55481.2022.9888839
12. Zhulkovskyi Oleg, Zhulkovska Inna, Vokhmianin Hlib, Tkach Anastasiia (2025). Comparative Analysis of computational performance of modern programming languages | *Computer system and information technologies* | *Dniprovsky State Technical University* | https://doi.org/10.31891/csit-2025-2-12
13. Swacha, J., & Muszyńska, K. (2011). Python and C#: A comparative analysis from Students' perspective. *Annales Universitatis Mariae Curie-Skłodowska. Sectio AI, Informatica*, *Vol. 11*(1). http://yadda.icm.edu.pl/baztech/element/bwmeta1.element.baztech-8d8c5876-0852-4acd-8ad7-89aed7c05523
14. *Definition of computer science | Dictionary.com*. (n.d.). Www.dictionary.com. https://www.dictionary.com/browse/computer-science
15. Belford, G. G., & Tucker, A. (2019). computer science. In *Encyclopædia Britannica*. https://www.britannica.com/science/computer-science
16. GeeksforGeeks. (2019, October 9). *Difference between Python and C#*. GeeksforGeeks. https://www.geeksforgeeks.org/python/difference-between-python-and-c-sharp/
17. *Computer Science as a Profession*. (2025). Archive.org. https://web.archive.org/web/20080617030847/http://www.csab.org/comp_sci_profession.html
18. Swed, K. (2023, October 30). *ComputerScience.org*. ComputerScience.org. https://www.computerscience.org/careers/software-engineer/
19. *ChatGPT*. (n.d.). ChatGPT. Retrieved 1 December 2025, from https://chatgpt.com/ *Explain the difference between C# and Python*
20. *Dataset from Kaggle, https://www.kaggle.com/datasets/likithagedipudi/linkedin-compatibility-dataset-50k-profiles?select=compatibility_pairs.csv*
21. *Bachkaniwala, R., Lanka, H., Rong, K., & Gavrilovska, A. (2024). Lotus: Characterization of Machine Learning Preprocessing Pipelines via Framework and Hardware Profiling. 2024 IEEE International Symposium on Workload Characterization (IISWC), 30–43. https://doi.org/10.1109/IISWC63097.2024.00013*

22. *Edwards, R. E., Zhang, H., Parker, L. E., & New, J. R. (2013). Approximate l-Fold Cross-Validation with Least Squares SVM and Kernel Ridge Regression. 2013 12th International Conference on Machine Learning and Applications (ICMLA), https://doi.org/10.1109/ICMLA.2013.18*
23. *Salihu, S., & Tafa, Z. (2020). On Computational Performances of the Actual Image Classification Methods in C# and Python. 2020 9th Mediterranean Conference on Embedded Computing (MECO). https://doi.org/10.1109/MECO49872.2020.9134248*
24. *Buschjäger, S., & Morik, K. (2018). Decision Tree and Random Forest Implementations for Fast Filtering of Sensor Data. IEEE Transactions on Circuits and Systems I: Regular Papers, 65(1), 209–222. https://doi.org/10.1109/TCSI.2017.2710627*
25. *N. Girish Chandra Choudhary, T. Rajesh Kumar, Vishwanathan Nagarajan, A. Rajalingam (2024). Road Accidents Prediction Using Decision Tree in Comparison to Linear Regression. 2024 IEEE 9th International Conference on Engineering Technologies and Applied Sciences (ICETAS) https://ieeexplore-ieee-org.ezproxy.hkr.se/document/11120049*
26. *Y. Zhu. (2024). Research on Taobao Big Data Analysis Based on Machine Learning and its Application in E-Commerce. 2024 6th International Conference on Applied Machine Learning (ICAML) https://ieeexplore-ieee-org.ezproxy.hkr.se/document/11344574*
27. *A. Ahmad, X. Sun, M. R. Naeem, Y. Javed, M. Akour and K. Sandahl (2025). "Understanding Flaky Tests Through Linguistic Diversity: A Cross-Language and Comparative Machine Learning Study,". IEEE Access, vol. 13, pp. 54561-54584. https://ieeexplore-ieee-org.ezproxy.hkr.se/document/10937160*
28. *Zhang, Q., Che, X., Chen, Y., Ma, X., Xu, M., Dustdar, S., Liu, X., & Wang, S. (2024). A Comprehensive Deep Learning Library Benchmark and Optimal Library Selection. IEEE Transactions on Mobile Computing, 23(5), 5069–5082. https://doi.org/10.1109/TMC.2023.3301973*
29. *Reddi, V. J., Cheng, C., Kanter, D., Mattson, P., Schmuelling, G., Wu, C.-J., Anderson, B., Breughe, M., Charlebois, M., Chou, W., Chukka, R., Coleman, C., Davis, S., Deng, P., Diamos, G., Duke, J., Fick, D., Gardner, J. S., Hubara, I., Idgunji, S., Jablin, T. B., Jiao, J., John, T. S., Kanwar, P., Lee, D., Liao, J., Lokhmotov, A., Massa, F., Meng, P., Micikevicius, P., Osborne, C., Pekhimenko, G., Rajan, A. T. R., Sequeira, D., Sirasao, A., Sun, F., Tang, H., Thomson, M., Wei, F., Wu, E., Xu, L., Yamada, K., Yu, B., Yuan, G., Zhong, A., Zhang, P., & Zhou, Y. (2020). MLPerf Inference Benchmark. 2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA), 446–459. https://doi.org/10.1109/ISCA45697.2020.00045*
30. *Zhang, C., Zhang, F., Guo, X., He, B., Zhang, X., & Du, X. (2021). iMLBench: A Machine Learning Benchmark Suite for CPU-GPU Integrated Architectures. IEEE Transactions on Parallel and Distributed Systems, 32(7), 1740–1752. https://doi.org/10.1109/TPDS.2020.3046870*
31. *Cheng, Y., Li, D., Guo, Z., Jiang, B., Geng, J., Bai, W., Wu, J., & Xiong, Y. (2021). Accelerating End-to-End Deep Learning Workflow With Codesign of Data Preprocessing and Scheduling. IEEE Transactions on Parallel and Distributed Systems, 32(7), 1802–1814. https://doi.org/10.1109/TPDS.2020.3047966*
32. *Paganelli, M., Sottovia, P., Park, K., Interlandi, M., & Guerra, F. (2023). Pushing ML Predictions Into DBMSs. IEEE Transactions on Knowledge and Data Engineering, 35(10), 10295–10308.*

https://doi.org/10.1109/TKDE.2023.3269592

9. Appendices

9.1 List of Acronyms

- CNN : Convolutional Neural Network
- CLR : Common Language Runtime
- CNTK : Microsoft Cognitive Toolkit
- CPU : Central Processing Unit
- CSV : Comma-Separated Values
- FPGA : Field- Programmable Gate Array
- GB : Gigabyte
- IEEE : institute of Electrical and Electronics Engineers
- IL : Intermediate Language
- JIT : Just In Time
- ML : Machine Learning
- ML.NET : Machine Learning for .NET
- MNIST : Modified National Institute of Standards and Technology database
- NCNN : Neural Network Compression Framework
- NumPy ; Numerical Python
- OS : Operating System
- Ram : Random Access Memory
- RQ : Research Question
- SIMD : Single Instruction, Multiple Data
- SSD : Solid State Drive

9.2 Raw Data from Experiments